

Real-world (a real approval pause) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real human-in-the-loop system handles the messy edge cases). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You already learned `interrupt()` in Doc09 — it pauses the graph completely and hands control back to whoever is running it, until they resume it with an answer.

This exercise wants you to use it for a specific, real reason: before the graph does anything based on what it found in your documents, stop and show a human what it found, and wait for a yes/no.

The key detail: what you pause with (the draft answer and its sources) needs to actually be shown to the person approving — not just "type yes to continue" with no context.

Things to use:

- `from langgraph.types import interrupt, Command`
- A node that calls `interrupt({...})` with the draft and sources.
- A checkpoint (from Doc09) attached to your graph, so the pause is actually saved, not lost.
- `graph.invoke(Command(resume="approved"), config)` to continue after the pause, using the same `thread_id` as the first call.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

`interrupt(value)` from `langgraph.types` pauses graph execution and surfaces `value` to whoever is running the graph. Execution stops entirely at that point — nothing after it runs until you call the graph again with a `Command(resume=...)`.

```
from langgraph.types import interrupt

def approval_node(state: dict) -> dict:
    decision = interrupt({"draft": state["draft"], "sources": state["sources"]})
    ...
```

The exact pieces:

- **`interrupt(value)`** — raises a special internal signal LangGraph catches; it isn't a normal Python exception you handle yourself. The graph run genuinely stops there. The dict you pass is exactly what the human reviewing sees, so it needs to carry both the draft answer *and* the source chunks it was grounded in, per this document's Core Concepts point about state needing to carry what was found, not just the original question.
- **The checkpointer** — without one attached to your graph (`graph.compile(checkpointer=...)`), there's nothing to resume *from*. This is exactly why Doc09's saved state matters here, not just conceptually.
- **`Command(resume=value)`** — the `value` passed here is what `interrupt()`'s call site receives back as its return value, letting the paused node continue with the human's actual decision, not just a generic "continue" signal.
- **The same `thread_id`** — both the first `invoke()` and the later resumed `invoke()` need the same `config={"configurable": {"thread_id": "..."}}`, or LangGraph has no saved state to resume from.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Every example so far assumes someone resumes the graph promptly. Real approvals don't work that way — a human might be in a meeting, asleep, or on vacation, and the paused thread just sits there, indefinitely, taking up a slot in whatever storage your checkpointer uses. And when they finally do act, "approved" or "rejected" by itself doesn't say *who* decided, or *when* — which matters the moment anything goes wrong and someone asks "who approved this?"

The real design question isn't just "how do I pause and resume" — it's "what happens if nobody resumes this in a reasonable time, and how do I keep a record of who made the call, not just what they decided?"

Two extra pieces answer that:

- **An audit-friendly resume value** — instead of resuming with a bare string like `"approved"`, resume with a small dict: `Command(resume={"decision": "approved", "by": "jane@example.com", "at": "2026-09-10T14:30:00Z"})`. The interrupted node reads `decision` for its logic exactly as before, but the full record — who, when — is now part of what actually happened, not lost the moment the terminal closes.
- **A staleness check** — before actually treating an old paused thread's resume as valid, compare the interrupt's original timestamp (stored alongside the `interrupt()` payload) against now. If it's been paused far longer than any real reviewer should reasonably take (a week, say), the resuming code should flag that explicitly ("this approval is stale — the source documents may have changed since this was searched") rather than silently trusting a decision made against possibly-outdated context.

```
from datetime import datetime, timezone

def approval_node(state: dict) -> dict:
    paused_at = datetime.now(timezone.utc).isoformat()
    decision = interrupt({
        "draft": state["draft"],
        "sources": state["sources"],
        "paused_at": paused_at,
    })
    return {
        "approved": decision["decision"] == "approved",
        "approved_by": decision.get("by"),
        "approved_at": decision.get("at"),
    }
```

Sketch the staleness check yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both treat "pause, then resume" as a clean, prompt round trip — correct for a demo, but a demo assumes a human is right there ready to answer. Advanced accounts for the gap between "paused" and "resumed" actually mattering: who made the decision needs to be recorded, not just what they decided, and a decision made a long time after the pause deserves a second look before being trusted blindly — the same "don't silently act on possibly-stale grounding" concern this document raises for search results, applied here to the approval itself.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
function approval_node(state):
    show the draft answer and where it came from
    call interrupt with that information
    decision = whatever comes back when resumed
    save the decision onto the state
    return the updated state

run it the first time:
    graph stops right at approval_node, waiting

resume it:
    call the graph again, same conversation id, with Command(resume="approved")
    check the graph continues from exactly where it paused
```

Here's almost the whole thing — just try running it and reading it line by line:

```
from langgraph.types import interrupt, Command

def approval_node(state):
    decision = interrupt({"draft": state["draft"], "sources": state["sources"]})
    return {"approved": decision == "approved"}

config = {"configurable": {"thread_id": "thread-1"}}
graph.invoke({"task": "some task"}, config)           # pauses here
graph.invoke(Command(resume="approved"), config)      # resumes and finishes
```

Expected output if you run just this: the first `invoke()` returns without an `"approved"` key at all — the run stopped inside `approval_node`, before it returned anything. The second `invoke()` is what actually produces `{"approved": True, ...}` in the final state.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
define:
    def approval_node(state: dict) -> dict:
        decision = interrupt({"draft": state["draft"], "sources": state["sources"]})
        return {"approved": decision == "approved"}

compile the graph with a checkpointer:
    graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "test-thread-1"}}

first call:
    graph.invoke({"task": "..."}, config)
    -> execution pauses at approval_node, returns control here

resume call:
    graph.invoke(Command(resume="approved"), config)
    -> approval_node's interrupt() call returns "approved", graph continues
```

```
from langgraph.types import interrupt, Command

def approval_node(state: dict) -> dict:
    decision = interrupt({"draft": state["draft"], "sources": state["sources"]})
    return {"approved": decision == "approved"}
```

Compile the graph with a real checkpointer, and write the two-call pause/resume test with a shared `thread_id`, testing both the `"approved"` and `"rejected"` resume values, then compare against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
function approval_node(state):
    paused_at = current UTC time, as a string
    decision = interrupt with draft, sources, and paused_at

    approved = decision's "decision" field equals "approved"
    return approved flag, plus who decided and when, from the decision dict

function is_stale(paused_at, max_age_hours):
    parse paused_at back into a datetime
    if (now - paused_at) is longer than max_age_hours: return True
    return False
```

Here's almost the whole thing — fill in the missing piece yourself:

```
from datetime import datetime, timedelta, timezone
from langgraph.types import interrupt, Command

MAX_APPROVAL_AGE_HOURS = 168 # 1 week

def approval_node(state: dict) -> dict:
    paused_at = datetime.now(timezone.utc).isoformat()
    decision = interrupt({
        "draft": state["draft"],
        "sources": state["sources"],
        "paused_at": paused_at,
    })
    return {
        "approved": decision["decision"] == "approved",
        "approved_by": decision.get("by"),
        "approved_at": decision.get("at"),
    }

def is_stale(paused_at: str, max_age_hours: int = MAX_APPROVAL_AGE_HOURS) -> bool:
    paused_time = datetime.fromisoformat(paused_at)
    # your turn: compare how much time has passed between paused_time and
    # datetime.now(timezone.utc) against max_age_hours, and return True if
    # it's been longer -- this is what a resuming caller should check
```

```
# *before* trusting an old interrupt's decision  
...
```

Fill in `is_stale` 's comparison yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate's pause/resume both trust the resume value blindly and record nothing about who decided or how long the pause lasted. Advanced records both — a structured resume payload naming who approved and when, and an explicit staleness check so a decision made against possibly-outdated search results gets flagged instead of silently trusted, which matters the moment "who approved this, and was the information still current?" becomes a real question someone asks.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)